# Algorithms

GATE CSE 2027 · complete formula, rule, method & trap sheet

**50** reference boxes · **9** chapters ·

Asymptotic Analysis & Recurrences | Searching & Sorting | Divide & Conquer | Greedy Algorithms | Dynamic Programming | Graph Algorithms | Amortized Analysis, Hashing & NP-Completeness | Problem → Standard Method Playbook | Last-minute one-liners

## 1 Asymptotic Analysis & Recurrences

### Asymptotic notation

REFERENCE TABLE

| Notation | Meaning | Read as |
|---|---|---|
| $f = O(g)$ | $f(n) \le c \cdot g(n)$ for all $n \ge n_0$ | g is an **upper** bound |
| $f = \Omega(g)$ | $f(n) \ge c \cdot g(n)$ for all $n \ge n_0$ | g is a **lower** bound |
| $f = \Theta(g)$ | both $O(g)$ and $\Omega(g)$ | **tight** bound (same growth rate) |
| $f = o(g)$ / $\omega(g)$ | $f/g \to 0$ / $f/g \to \infty$ | **strictly** smaller / larger |

RULES & FACTS

- Constants and lower-order terms are dropped: $5n^2 + 3n \log n = \Theta(n^2)$. The base of a logarithm only changes a constant factor, so $\log_2 n = \Theta(\log_{10} n)$; but $2^n$ and $4^n$ are **not** $\Theta$ of each other (the base of an exponential matters).
- **Worst, average and best case** are different functions of n; O/Ω/Θ describe bounds on whichever function you chose. "Insertion sort is Ω(n)" (best case) and "$O(n^2)$" (worst case) are both true.
- Transitivity holds for O, Ω, Θ; Θ is symmetric; $f = O(g) \Leftrightarrow g = \Omega(f)$.

### Growth-rate ladder

FORMULA / WORKING

$$1 \;<\; \log\log n \;<\; \log n \;<\; (\log n)^k \;<\; n^{\varepsilon} \;<\; n \;<\; n \log n \;<\; n^2 \;<\; n^3 \;<\; 2^n \;<\; n! \;<\; n^n$$

RULES & FACTS

- Any polynomial beats any polylog; any exponential beats any polynomial.
- $\mathbf{n^{\log n} = 2^{(\log n)^2}}$ grows faster than every polynomial but slower than $2^n$, because $(\log n)^2 = o(n)$.
- **$\log(n!) = \Theta(n \log n)$** (Stirling). Also $\sqrt{n}$ is much bigger than $\log n$; $n^{0.01}$ beats $(\log n)^{100}$ eventually.
- Compare functions by taking logs or by limits: $\lim f/g = 0 \Rightarrow f = o(g)$.

### Analysing loops

WORKED EXAMPLE

**Worked numerical — nested loop with a doubling outer index**

```
for (i = 1; i <= n; i *= 2)
    for (j = 1; j <= i; j++)
        count++;
i takes the values 1, 2, 4, …, 2^k  with k = ⌊log₂ n⌋
count = 1 + 2 + 4 + … + 2^k = 2^(k+1) − 1  <  2n
n = 16 → 31       n = 100 → 1 + 2 + 4 + 8 + 16 + 32 + 64 =
127       →  Θ(n), not Θ(n log n)
```

RULES & FACTS

- `for (i = 1; i < n; i *= 2)` runs $\lceil \log_2 n \rceil$ times. `for (i = 1; i*i <= n; i++)` runs $\lfloor \sqrt{n} \rfloor$ times ($n = 100 \to 10$).
- Nested independent loops multiply: $n \times (\log n) \times \ldots$ ; when the inner bound depends on the outer index, **sum** the series instead.
- For `for (i = n; i > 1; i = √i)` the count is $\Theta(\log \log n)$.
- Space complexity counts extra memory beyond the input, including the **recursion stack depth**.

### Solving recurrences

#### Master theorem for $T(n) = a \cdot T(n/b) + f(n)$, $a \ge 1$, $b > 1$

NOTES

Let $n^c = n^{\log_b a}$ (the work done at the leaves). Compare f(n) with it:

REFERENCE TABLE

| Case | Condition | Result |
|---|---|---|
| 1 | $f(n) = O(n^{c-\varepsilon})$ — leaves dominate | $T(n) = \Theta(n^c)$ |
| 2 | $f(n) = \Theta(n^c \log^k n)$, $k \ge 0$ — balanced | $T(n) = \Theta(n^c \log^{k+1} n)$ |
| 3 | $f(n) = \Omega(n^{c+\varepsilon})$ and $a \cdot f(n/b) \le \delta \cdot f(n)$, $\delta < 1$ — root dominates | $T(n) = \Theta(f(n))$ |

REFERENCE TABLE

| Recurrence | $n^{\log_b a}$ | Case | Solution |
|---|---|---|---|
| $T(n) = 2T(n/2) + n$ *(merge sort)* | $n$ | 2 | **$\Theta(n \log n)$** |
| $T(n) = 4T(n/2) + n$ | $n^2$ | 1 | $\Theta(n^2)$ |
| $T(n) = 4T(n/2) + n^2$ | $n^2$ | 2 | **$\Theta(n^2 \log n)$** |
| $T(n) = 3T(n/2) + n^2$ | $n^{1.58}$ | 3 | $\Theta(n^2)$ |
| $T(n) = T(n/2) + 1$ *(binary search)* | 1 | 2 | $\Theta(\log n)$ |
| $T(n) = 2T(n/2) + 1$ | $n$ | 1 | $\Theta(n)$ |
| $T(n) = 2T(n/2) + n \log n$ | $n$ | 2 ($k = 1$) | **$\Theta(n \log^2 n)$** |
| $T(n) = 7T(n/2) + n^2$ *(Strassen)* | $n^{2.81}$ | 1 | $\Theta(n^{\log_2 7})$ |

COMMON TRAP

**When the Master theorem does not apply**

a or b not constant, f(n) not polynomially related (e.g. n/log n), or subproblems of unequal size such as T(n/3) + T(2n/3). Use the recursion tree, substitution, or the Akra–Bazzi idea instead.

### Other forms

REFERENCE TABLE

| Recurrence | Solution | How |
|---|---|---|
| $T(n) = T(n-1) + n$ | $\Theta(n^2)$ | unroll: $n + (n-1) + \dots + 1$ |
| $T(n) = T(n-1) + 1$ | $\Theta(n)$ | unroll |
| $T(n) = 2T(n-1) + 1$ | $\Theta(2^n)$ | $T(n) + 1$ doubles each step (Tower of Hanoi: $2^n - 1$) |
| $T(n) = T(n-1) + T(n-2) + 1$ | $\Theta(\varphi^n)$ | Fibonacci-type, exponential |
| $T(n) = T(\sqrt{n}) + 1$ | $\Theta(\log \log n)$ | $n = 2^{(2^k)}$ takes k steps |
| $T(n) = T(n/3) + T(2n/3) + n$ | $\Theta(n \log n)$ | recursion tree: every level costs $\le n$, depth $\log_{3/2} n$ (checked numerically) |
| $T(n) = T(n/5) + T(7n/10) + n$ | $\Theta(n)$ | sizes sum to $9n/10 < n$: geometric decrease (median of medians) |

WORKED EXAMPLE

**Worked numerical — a Case 2 recurrence with a log factor**

**Q:** Solve $T(n) = 2T(n/2) + n \log n$.

**A:** $n^{\log_2 2} = n$ and $f(n) = n \cdot \log^1 n$, so $k = 1$ and $T(n) = \Theta(n \log^2 n)$. A numerical evaluation of the recurrence confirms that $T(n)/(n \log^2 n)$ settles near a constant ($\approx 0.53$–$0.56$ for $n = 2^{10} \dots 2^{18}$).

## 2 Searching & Sorting

### Searching

RULES & FACTS

- **Linear search**: $\Theta(n)$ worst case, $(n+1)/2$ comparisons on average for a successful search.
- **Binary search** on a sorted array: at most $\lfloor \log_2 n \rfloor + 1$ comparisons ($n = 15 \to 4$, $n = 1000 \to 10$ because $2^{10} = 1024$). Recurrence $T(n) = T(n/2) + 1$. Not possible in $O(\log n)$ on a linked list.
- Interpolation search: $O(\log \log n)$ on uniformly distributed data, $O(n)$ worst case. Hash search: $O(1)$ average.

### Comparison sorts at a glance

REFERENCE TABLE

| Algorithm | Best | Average | Worst | Extra space | Stable? |
|---|---|---|---|---|---|
| Bubble | n (with early exit) | $n^2$ | $n^2$ | 1 | Yes |
| Selection | $n^2$ | $n^2$ | $n^2$ | 1 | No |
| Insertion | **n** | $n^2$ | $n^2$ | 1 | Yes |
| Merge | n log n | n log n | n log n | O(n) | Yes |
| Quick | n log n | n log n | **$n^2$** | O(log n) stack (O(n) worst) | No |
| Heap | n log n | n log n | n log n | 1 | No |

NOTES

A sort is **stable** if equal keys keep their original relative order, and **in place** if it uses O(1) extra memory. Selection sort makes at most $n-1$ swaps (the fewest of the simple sorts); insertion sort is best for nearly sorted data and small arrays; merge sort is the choice for linked lists and external sorting.

FORMULA / RULE

**Useful exact counts**

```
merge two sorted lists (m, n): m + n − 1 worst
merge sort, n = 2^k:  n log₂n − n + 1  worst
selection sort comparisons: n(n−1)/2
max AND min together:  ⌈3n/2⌉ − 2
second largest:       n + ⌈log₂n⌉ − 2
```

### Lower bound and non-comparison sorts

RULES & FACTS

- Any comparison sort corresponds to a binary **decision tree** with at least $n!$ leaves, so its height is $\ge \lceil \log_2 n! \rceil = \Omega(n \log n)$. Sorting 5 items needs at least $\lceil \log_2 120 \rceil = \mathbf{7}$ comparisons in the worst case.
- **Counting sort**: keys are integers in $0 \dots k$; $\Theta(n + k)$ time and space, stable. **Radix sort**: d passes of a stable sort, $\Theta(d(n + k))$. **Bucket sort**: $\Theta(n)$ expected for uniform input. These beat $n \log n$ only because they do not compare keys.

WORKED EXAMPLE

**Worked numerical — inversions and shifts**

```
[2, 4, 1, 3, 5]  : inversions (2,1) (4,1) (4,3) = 3  → insertion sort makes 3 shifts
[3, 8, 2, 5, 1, 4, 7, 6] : 12 inversions → 12 shifts, and bubble sort makes 12 swaps
[5, 4, 3, 2, 1] : 10 inversions = 5·4/2 (the maximum), bubble sort needs 10 swaps
```

### Counting operations

FORMULA / RULE

**Inversions**

A pair $i < j$ with $a[i] > a[j]$. Maximum $n(n-1)/2$ (reverse sorted). Insertion sort performs **exactly one shift per inversion**; bubble sort performs exactly one swap per inversion.

WORKED EXAMPLE

**Worked numerical — quicksort worst case**

With the first (or last) element as the pivot, an already sorted array splits into 0 and $n-1$ elements every time, so comparisons $= (n-1) + (n-2) + \dots + 1 = n(n-1)/2$: $n = 6 \to \mathbf{15}$, $n = 8 \to 28$ (counted by running the algorithm). The recurrence is $T(n) = T(n-1) + n$, i.e. $\Theta(n^2)$. **Randomised pivot** or **median-of-three** avoids this on sorted input; expected comparisons $\approx 2n \ln n \approx 1.39\, n \log_2 n$.

WORKED EXAMPLE

**Worked numerical — merge sort comparisons (n = 8)**

Worst case = 8·3 − 8 + 1 = **17** comparisons, best case = 12 ($n/2 \cdot \log_2 n$). Both values were confirmed by sorting all 8! = 40 320 permutations and recording the minimum and maximum comparison counts.

### Selection problems

RULES & FACTS

- **Quickselect** (k-th smallest): partition around a pivot and recurse into one side — O(n) average, $O(n^2)$ worst.
- **Median of medians** guarantees a good pivot (groups of 5), giving O(n) worst case: T(n) = T(n/5) + T(7n/10) + O(n).
- Finding the maximum needs exactly n − 1 comparisons. Finding max and min together takes $\lceil 3n/2 \rceil - 2$ (compare in pairs first): n = 8 → 10, n = 100 → **148** (verified by simulation).

GATE FAVOURITE

**GATE angle**

Know which sort is stable, in place, adaptive (insertion), and which has an $O(n^2)$ worst case (quick, insertion, bubble, selection). Expect questions on **number of swaps/comparisons for a given array**, **effect of the pivot rule** and "which sorting algorithm will take the least time on an already sorted input" (insertion sort: Θ(n)).

## 3 Divide & Conquer

### The pattern

RULES & FACTS

1. **Divide** the input into a smaller instances of size n/b.
2. **Conquer** by solving them recursively (base case for tiny inputs).
3. **Combine** the answers in f(n) time.

REFERENCE TABLE

| Algorithm | Recurrence | Time |
|---|---|---|
| Binary search | T(n) = T(n/2) + 1 | Θ(log n) |
| Merge sort | T(n) = 2T(n/2) + n | Θ(n log n) |
| Quicksort (balanced split) | T(n) = 2T(n/2) + n | Θ(n log n); worst $\Theta(n^2)$ |
| Max & min of an array | T(n) = 2T(n/2) + 2 | 3n/2 − 2 comparisons (n a power of 2) |
| Maximum-subarray (D&C) | T(n) = 2T(n/2) + n | Θ(n log n) (Kadane's DP does Θ(n)) |
| Closest pair of points | T(n) = 2T(n/2) + n | Θ(n log n) |
| Karatsuba multiplication | T(n) = 3T(n/2) + n | $\Theta(n^{\log_2 3}) \approx n^{1.585}$ |
| Strassen matrix multiplication | T(n) = 7T(n/2) + $n^2$ | $\Theta(n^{\log_2 7}) \approx n^{2.81}$ (naive: $n^3$) |
| Median of medians selection | T(n) = T(n/5) + T(7n/10) + n | Θ(n) |
| Fast exponentiation | T(n) = T(n/2) + 1 | Θ(log n) multiplications |

### Worked ideas

WORKED EXAMPLE

**Worked numerical — Strassen**

Naive 2×2 block multiplication needs 8 recursive products, giving $n^3$. Strassen uses 7 products plus extra additions ($\Theta(n^2)$ work).

```
fully recursing an 8×8 matrix down to 1×1 scalars:
   naive:    8³ = 512 scalar multiplications
   Strassen: 7³ = 343 scalar multiplications          (4×4: 49 vs 64)
exponent log₂7 ≈ 2.807  vs  3
```

WORKED EXAMPLE

**Worked numerical — fast exponentiation (square and multiply)**

$x^{13}$: $13 = 1101_2$. Squarings needed = $\lfloor \log_2 13 \rfloor = 3$ and extra multiplications = (number of 1 bits) − 1 = 2, so **5** multiplications instead of 12. $x^{15}$ needs 6, $x^{16}$ needs only 4.

WORKED EXAMPLE

**Worked numerical — max and min with n = 8**

Recurrence T(2) = 1, T(n) = 2T(n/2) + 2 → T(n) = 3n/2 − 2 = **10** for n = 8. Compare this with the straightforward 2(n − 1) = 14 comparisons.

RULES & FACTS

- **Merge step of merge sort** uses O(n) extra space; sorting an array of n elements has recursion depth $\log_2 n$.
- **Counting inversions** in O(n log n): piggy-back on merge sort — when an element from the right half is merged before elements of the left half, it forms inversions with all remaining left elements.
- Efficiency of D&C comes from the combine cost: if the subproblems overlap (e.g. Fibonacci), plain D&C is exponential and **dynamic programming** is needed instead.

## 4 Greedy Algorithms

### When greedy works

RULES & FACTS

- **Greedy-choice property**: some optimal solution contains the greedy first choice (proved by an **exchange argument**: swap any optimal solution's first choice for the greedy one without making it worse).
- **Optimal substructure**: after the greedy choice, the rest is a smaller instance of the same problem.
- Greedy algorithms are usually simple and fast (often O(n log n) because of a sort or a heap) but can be **wrong**: always look for a counter-example.

### Classic problems

REFERENCE TABLE

| Problem | Greedy rule | Time |
|---|---|---|
| Activity selection (maximum number of non-overlapping activities) | sort by **earliest finish time**; take each activity that starts after the last chosen one ends | O(n log n) |
| Fractional knapsack | sort by value/weight ratio; take whole items, then a fraction of the next | O(n log n) |
| Huffman coding | repeatedly merge the two least-frequent nodes | O(n log n) with a min-heap |
| Optimal merge pattern | always merge the two smallest files (same as Huffman) | O(n log n) |
| Job sequencing with deadlines | sort by profit; place each job in the latest free slot ≤ its deadline | $O(n^2)$ (O(n log n) with union-find) |
| Minimising total waiting time | shortest job first | O(n log n) |
| MST (Kruskal, Prim) | cheapest safe edge | see graph section |
| Dijkstra | closest unvisited vertex | see graph section |

WORKED EXAMPLE

**Worked numerical — activity selection**

Activities (start, finish): (1,4) (3,5) (0,6) (5,7) (3,9) (5,9) (6,10) (8,11) (8,12) (2,14) (12,16). Sorted by finish time, the greedy picks **(1,4), (5,7), (8,11), (12,16)** — 4 activities, which is the maximum. Picking the shortest or the earliest-starting activity first is **not** always optimal.

WORKED EXAMPLE

**Worked numerical — Huffman coding**

**Q:** Frequencies (per 100 characters): a 45, b 13, c 12, d 16, e 9, f 5. Find the total encoded length.

**A:** Merge the two smallest repeatedly: f+e = 14; c+b = 25; 14+d = 30; 25+30 = 55; a+55 = 100.

```
code lengths:  a = 1,  b = 3,  c = 3,  d = 3,  e = 4,  f = 4
total bits = 45·1 + 13·3 + 12·3 + 16·3 + 9·4 + 5·4 = 45 + 39
+ 36 + 48 + 36 + 20 = 224
fixed-length 3-bit code: 300 bits.   (also = sum of all
internal-node weights: 14 + 25 + 30 + 55 + 100 = 224)
```

A Huffman tree on n symbols has n − 1 internal nodes; Huffman codes are **prefix-free**, and the two least frequent symbols end up as siblings at the deepest level.

WORKED EXAMPLE

**Worked numerical — knapsack: fractional vs 0/1**

Items (value, weight): (60, 10), (100, 20), (120, 30); capacity 50. Ratios: 6, 5, 4.

```
Fractional (greedy by ratio): all of item 1 (60) + all of
item 2 (100) + 20/30 of item 3 (80) = 240
0/1 greedy by ratio: items 1 and 2 → weight 30, value 160
(item 3 does not fit whole)
0/1 optimum (dynamic programming): items 2 and 3 → weight
50, value 220
```

So greedy is optimal for the fractional version but **fails for 0/1 knapsack**, which needs DP.

WORKED EXAMPLE

**Worked numerical — coin change where greedy fails**

Coins {1, 3, 4}, amount 6: greedy takes 4 + 1 + 1 = **3** coins, but 3 + 3 = **2** coins is optimal. Greedy is correct for canonical systems such as {1, 5, 10, 25} and for {1, 2, 5, 10}.

WORKED EXAMPLE

**Worked numerical — job sequencing with deadlines**

Jobs (profit, deadline): A (100, 2), B (19, 1), C (27, 2), D (25, 1), E (15, 3). Process in decreasing profit: A → slot 2; C → slot 1 (latest free ≤ 2); D and B need slot 1 (taken) → rejected; E → slot 3. Schedule C, A, E with profit 27 + 100 + 15 = **142**.

WORKED EXAMPLE

**Worked numerical — optimal merge pattern**

Merging files of sizes 20, 30, 10, 5, 30 pairwise with minimum total cost: 5+10 = 15; 15+20 = 35; 30+30 = 60; 35+60 = 95 → total cost 15 + 35 + 60 + 95 = **205**.

## 5 Dynamic Programming

### Method

RULES & FACTS

1. Define the subproblem (the state) and what the table entry means.
2. Write the recurrence that builds a state from smaller ones, plus the base cases.
3. Fill the table in an order that respects dependencies (**tabulation**, bottom-up) or recurse with a cache (**memoisation**, top-down).
4. Time = (number of states) × (work per state); space = table size (often reducible to one or two rows).

NOTES

**DP vs divide & conquer**: D&C subproblems are independent; DP subproblems overlap. **DP vs greedy**: DP examines all choices (guaranteed optimal when the structure holds); greedy commits to one. Naive recursion on overlapping subproblems is exponential — Fibonacci with memoisation drops from $\Theta(\varphi^n)$ to $\Theta(n)$.

### Standard problems

REFERENCE TABLE

| Problem | Recurrence | Time / space |
|---|---|---|
| **Longest common subsequence** (strings of length m, n) | L[i][j] = L[i−1][j−1] + 1 if $x_i = y_j$, else max(L[i−1][j], L[i][j−1]) | Θ(mn) time; (m+1)(n+1) table |
| **Edit distance** | D[i][j] = min(D[i−1][j] + 1, D[i][j−1] + 1, D[i−1][j−1] + [$x_i \neq y_j$]) | Θ(mn) |
| **0/1 knapsack** (n items, capacity W) | K[i][w] = max(K[i−1][w], $v_i$ + K[i−1][w − $w_i$]) | Θ(nW) — *pseudo-polynomial* |
| **Matrix-chain multiplication** | M[i][j] = $\min_{i \le k < j}$ (M[i][k] + M[k+1][j] + $p_{i-1}p_kp_j$) | Θ($n^3$) time, Θ($n^2$) space |
| **Longest increasing subsequence** | L[i] = 1 + max L[j] over j < i with a[j] < a[i] | Θ($n^2$); Θ(n log n) with binary search |
| Coin change (minimum coins) | C[x] = 1 + min C[x − c] | Θ(amount · #coins) |
| Coin change (number of ways) | for each coin, W[x] += W[x − c] | Θ(amount · #coins) |
| Subset sum / partition | reachable[s] \|= reachable[s − $a_i$] | Θ(n · sum) |
| Floyd–Warshall (all-pairs shortest path) | $d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$ | Θ($V^3$) |
| Rod cutting, optimal BST, Catalan numbers, longest palindromic subsequence | same pattern: states over intervals or prefixes | |

WORKED EXAMPLE

**Worked numerical — LCS table**

**Q:** Find the length of the LCS of `ABCB` and `BDCAB`.

```
      ε  B  D  C  A  B
ε     0  0  0  0  0  0
A     0  0  0  0  1  1
B     0  1  1  1  1  2
C     0  1  1  2  2  2
B     0  1  1  2  2  3
```

The bottom-right entry is **3** (the subsequence B C B). Strings `AGGTAB` and `GXTXAYB` have an LCS of length **4** (G T A B); the CLRS strings `ABCBDAB` and `BDCABA` also give 4. A recursive solution without memoisation is exponential in the worst case, while the table needs Θ(mn).

WORKED EXAMPLE

**Worked numerical — edit distance**

`kitten → sitting`: substitute k→s, substitute e→i, insert g = **3** operations (`sunday → saturday` also takes 3: insert a, insert t, substitute n→r).

WORKED EXAMPLE

**Worked numerical — 0/1 knapsack**

Weights {1, 3, 4, 5}, values {1, 4, 5, 7}, capacity 7. The best choice is the items of weight 3 and 4 (value 4 + 5 = **9**). Taking weights 1 + 5 gives only 8, and 1 + 3 gives 5. Runtime Θ(nW) = 4 · 7 table cells.

WORKED EXAMPLE

**Worked numerical — matrix-chain multiplication**

```
A1 (10×30), A2 (30×5), A3 (5×60):
(A1A2)A3 : 10·30·5 + 10·5·60 = 1500 + 3000  = 4500
A1(A2A3) : 30·5·60 + 10·30·60 = 9000 + 18000 = 27000
Dimensions p = (40, 20, 30, 10, 30) for four matrices →
minimum = 26000 scalar multiplications
```

The number of ways to fully parenthesise n matrices is the Catalan number C(n−1); the table has about $n^2/2$ entries and each takes O(n).

WORKED EXAMPLE

**Worked numerical — LIS and coin change**

LIS of `[10, 22, 9, 33, 21, 50, 41, 60]` has length **5** (10, 22, 33, 50, 60). Minimum coins for 11 with {1, 2, 5} = **3** (5 + 5 + 1). Number of ways to make 5 with {1, 2, 5} = **4** (5; 2+2+1; 2+1+1+1; 1+1+1+1+1).

GATE FAVOURITE

**GATE angle**

Typical asks: the **time or space** of a DP, the **table size**, filling a few cells by hand, whether a problem has overlapping subproblems, and "which of these is solved by DP vs greedy". Remember that 0/1 knapsack and subset sum are **pseudo-polynomial** (polynomial in the numeric value W, not in its bit length), so they do not contradict NP-completeness.

## 6 Graph Algorithms

### Traversal: BFS and DFS

RULES & FACTS

- **BFS** (queue): visits vertices in increasing distance (number of edges) from the source; gives shortest paths in **unweighted** graphs; Θ(V + E) with adjacency lists, Θ($V^2$) with a matrix. A graph is **bipartite** iff BFS can 2-colour it (no odd cycle).
- **DFS** (stack / recursion): records discovery and finish times; Θ(V + E). Edge types in a directed graph: **tree, back, forward, cross**. A directed graph has a cycle **iff DFS finds a back edge**; an undirected graph has a cycle iff DFS meets a visited vertex that is not the parent.
- A DFS forest of an undirected graph with c components has V − c tree edges. **Articulation points** and **bridges** are found with DFS low-link values in Θ(V + E).

### Topological sort and SCC

RULES & FACTS

- A topological ordering (every edge u → v has u before v) exists **iff the graph is a DAG**. Two methods, both Θ(V + E): (1) **Kahn** — repeatedly remove a vertex of in-degree 0; (2) DFS — output vertices in **decreasing finish time**. The ordering is usually **not unique**.
- **Strongly connected components**: Kosaraju (DFS on G, then DFS on the transpose in decreasing finish time) or Tarjan (one DFS with low-links), both Θ(V + E). Contracting each SCC gives a DAG.

WORKED EXAMPLE

**Worked numerical — counting topological orderings**

DAG on vertices 0–5 with edges 5→2, 5→0, 4→0, 4→1, 2→3, 3→1. Kahn's algorithm with the smallest-index rule gives **4 5 0 2 3 1**. Counting all linear extensions (by testing every permutation of the 6 vertices) gives **13** valid topological orders.

WORKED EXAMPLE

**Worked numerical — SCCs**

Directed graph: 0→1, 1→2, 2→0, 2→3, 3→4, 4→5, 5→3, 6→5, 6→7, 7→6. SCCs: **{0,1,2}**, **{3,4,5}**, **{6,7}** — three components, and the condensation DAG is {6,7} → {3,4,5} ← {0,1,2}.

## Minimum spanning tree

RULES & FACTS

- An MST of a connected weighted graph has V − 1 edges and minimum total weight. If all edge weights are **distinct**, the MST is **unique**.
- **Cut property**: for any cut, the lightest crossing edge belongs to some MST. **Cycle property**: the heaviest edge on a cycle (if unique) belongs to no MST.
- **Kruskal**: sort edges, add the next edge if it joins two different components (union-find). Θ(E log E) = Θ(E log V).
- **Prim**: grow one tree from a start vertex, always adding the cheapest edge leaving it. Θ(E log V) with a binary heap; Θ(V²) with an adjacency matrix and no heap; Θ(E + V log V) with a Fibonacci heap.
- Adding the same constant to every edge, or squaring all positive weights, does not change which spanning tree is minimum (only the ordering of weights matters).

WORKED EXAMPLE

**Worked numerical — one graph, three algorithms**

Undirected graph on A–F with edges: A–B 4, A–C 2, B–C 1, B–D 5, C–D 8, C–E 10, D–E 2, D–F 6, E–F 3.

```
Kruskal (sorted: 1, 2, 2, 3, 4, 5, 5…):
  B–C (1) ✔   A–C (2) ✔   D–E (2) ✔   E–F (3) ✔   A–B (4) ✘ cycle   B–D (5) ✔ joins {A,B,C} with {D,E,F}
  MST weight = 1 + 2 + 2 + 3 + 5 = 13
Prim from A: A–C (2), C–B (1), B–D (5), D–E (2), E–F (3)  → also 13

Dijkstra from A:   A 0,  C 2,  B 3 (via C),  D 8 (via B),  E 10 (via D),  F 13 (via E)
Shortest path to F: A → C → B → D → E → F   with length 13
```

## Single-source shortest paths

REFERENCE TABLE

| Algorithm | Handles | Time |
|---|---|---|
| BFS | unweighted graphs | Θ(V + E) |
| **Dijkstra** | non-negative weights only | Θ((V + E) log V) with a binary heap; Θ(V²) with an array |
| **Bellman–Ford** | negative edges; **detects negative cycles** | Θ(VE) |
| DAG relaxation in topological order | negative edges, DAG only | Θ(V + E) |
| **Floyd–Warshall** | all pairs; negative edges but no negative cycle | Θ(V³), Θ(V²) space |
| Johnson's algorithm | all pairs on sparse graphs with negative edges | Θ(VE + V² log V) |

RULES & FACTS

- Relaxation: if d[u] + w(u, v) < d[v], set d[v] = d[u] + w(u, v). Bellman–Ford relaxes every edge V − 1 times; a further improving relaxation means a negative cycle.
- **Dijkstra fails with negative edges** because a vertex's distance is finalised when it is extracted, and a later negative edge could still shorten it.

WORKED EXAMPLE

**Worked numerical — Bellman–Ford with negative edges**

Directed graph, source s: s→a 6, s→b 7, a→b 8, a→c 5, a→d −4, b→c −3, b→d 9, c→a −2, d→s 2, d→c 7.

```
after V − 1 = 4 passes:  d(s) = 0,  d(a) = 2,  d(b) = 7,  d(c) = 4,  d(d) = −2
shortest path s → d : s → b → c → a → d, with length 7 + (−3) + (−2) + (−4) = −2 ✔
One more pass improves nothing → no negative cycle
```

WORKED EXAMPLE

**Worked numerical — Floyd–Warshall on 4 vertices**

Initial weights: 1→2 = 3, 1→4 = 7, 2→1 = 8, 2→3 = 2, 3→1 = 5, 3→4 = 1, 4→1 = 2 (others ∞). After the three nested loops (V³ = 64 updates):

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 3 | 5 | 6 |
| 2 | 5 | 0 | 2 | 3 |
| 3 | 3 | 6 | 0 | 1 |
| 4 | 2 | 5 | 7 | 0 |

For example d(1,4) improves from 7 to 6 via 1→2→3→4 (3 + 2 + 1).

GATE FAVOURITE

**GATE angle**

Know the **time complexity** of each algorithm (and its dependence on the data structure), **when it fails**, the order in which Dijkstra extracts vertices, and what the first few Kruskal/Prim steps pick. Floyd–Warshall detects a negative cycle by a negative value on the diagonal.

# 7 Amortized Analysis, Hashing & NP-Completeness

## Amortized analysis

NOTES

The amortized cost of an operation is the **average over a worst-case sequence** — no probability involved. Three methods: **aggregate** (total ÷ n), **accounting** (charge extra credit to cheap operations), **potential** (a potential function Φ ≥ 0, amortized cost = actual + ΔΦ).

WORKED EXAMPLE

**Worked numerical — dynamic array doubling**

Push onto an array whose capacity doubles when full. Copies happen at sizes 1, 2, 4, 8, … For n = 16 pushes: 16 writes + (1 + 2 + 4 + 8) = 15 copies = **31** total, i.e. under 2 per push. In general the total is < 3n, so the amortized cost per push is **O(1)** even though a single push can cost Θ(n).

WORKED EXAMPLE

**Worked numerical — binary counter**

Incrementing a k-bit counter flips many bits occasionally. Total flips over n increments from 0 equals 2n − (number of 1s in n): n = 8 → 15, n = 16 → **31**, n = 100 → 197. So the amortized cost per increment is **< 2** flips = O(1).

RULES & FACTS

- **Union–find** with union by rank and path compression: m operations cost O(m·α(n)), where α is the inverse Ackermann function (≤ 4 for any practical n).
- Stack with **multipop**, queue from two stacks, and splay trees are standard amortized examples.

## Hashing — analysis

RULES & FACTS

- With chaining and simple uniform hashing, load factor α = n/m: expected unsuccessful search Θ(1 + α), successful ≈ 1 + α/2. With α = O(1), operations are O(1) expected.
- Open addressing, uniform probing: expected probes ≈ 1/(1 − α) for an unsuccessful search and (1/α)·ln(1/(1 − α)) for a successful one.
- A **universal family** of hash functions makes the expected number of collisions small regardless of the input. The **birthday paradox**: collisions are likely once about √m keys are inserted. Cuckoo hashing gives worst-case O(1) lookups.

### P, NP and NP-completeness

REFERENCE TABLE

| Class | Definition |
|---|---|
| **P** | decision problems solvable in polynomial time by a deterministic algorithm |
| **NP** | problems whose "yes" answers can be **verified in polynomial time** given a certificate (equivalently, solvable by a non-deterministic polynomial-time machine). P ⊆ NP. |
| **NP-hard** | at least as hard as every problem in NP (every NP problem reduces to it); need not be in NP |
| **NP-complete** | in NP **and** NP-hard |

RULES & FACTS

- **Polynomial-time reduction** $A \leq_p B$: an easy translation of instances of A into instances of B. If B is easy then A is easy; if A is hard then B is hard. To prove B NP-hard, reduce a **known NP-complete** problem **to** B (direction matters!).
- **Cook–Levin theorem**: SAT is NP-complete (the first one). If any NP-complete problem is in P, then **P = NP**; if any is proved to need super-polynomial time, then P ≠ NP.

REFERENCE TABLE

| NP-complete | In P (polynomial time) |
|---|---|
| SAT, **3-SAT**, Clique, Independent Set, Vertex Cover, Hamiltonian cycle/path, Travelling Salesman (decision), Subset Sum, Partition, 3-colouring, 0/1 knapsack (decision), Set cover | **2-SAT**, shortest paths, MST, matching, maximum flow, 2-colouring (bipartiteness), Euler circuit, linear programming, primality testing (AKS), sorting, searching |

WORKED EXAMPLE

**Worked numerical — vertex cover and independent set are complements**

**Fact:** S is a vertex cover of G iff V − S is an independent set. So min vertex cover + max independent set = |V|. Check on the graph with edges 0–1, 1–2, 2–3, 3–4, 4–0, 0–5 (a 5-cycle with a pendant vertex): minimum vertex cover = 3 ({0, 2, 3} works), maximum independent set = 3 ({1, 3, 5}), and 3 + 3 = 6 = |V| (checked by brute force over all subsets).

RULES & FACTS

- **Pseudo-polynomial** algorithms run in time polynomial in the numeric *value* of the input (knapsack Θ(nW)); NP-complete problems that have them are only "weakly" NP-complete.
- **Approximation**: vertex cover has a simple 2-approximation (take both endpoints of a maximal matching); metric TSP has a 2-approximation (double the MST) and Christofides' 1.5.
- Relationship of classes believed true: P ⊂ NP ⊂ PSPACE ⊂ EXPTIME (P ≠ NP is an open question). Co-NP contains the complements of NP problems; P ⊆ NP ∩ co-NP.

GATE FAVOURITE

**GATE angle**

Typical questions: "which of these is not known to be NP-complete / is in P" (2-SAT, shortest path, MST are in P; 3-SAT, Clique, Hamiltonian cycle, Subset Sum are NPC), "what follows if X is in P", and which direction a reduction must go. The decision version of a problem is the one that is NP-complete; the optimisation version is NP-hard.

## 8 Problem → Standard Method Playbook

METHOD

### Heap Sort — canonical procedure

1. Build a **max-heap** for ascending order (bottom-up heapify, O(n)).
2. Swap the root with the last element of the active heap.
3. Reduce heap size by one and max-heapify the root.
4. Repeat until one element remains.

Time: **O(n log n)** in best/average/worst cases. Auxiliary array space: **O(1)** for in-place heap sort. It is not stable.

METHOD

### Open addressing — double hashing

For table size m, the standard probe sequence is:

`h(k,i) = (h₁(k) + i·h₂(k)) mod m`, i = 0,1,2,…

- Choose $h_2(k)$ relatively prime to m so the sequence can visit the entire table.
- Do not confuse double hashing with linear probing: linear probing adds a fixed step of 1.
- Deletion normally uses a tombstone/deleted marker; simply clearing a slot can break future searches.

METHOD

### BFS / DFS — recognition and invariants

- **BFS:** queue; explores by nondecreasing distance from the source. In an unweighted graph, first discovery gives a shortest-path distance.
- **DFS:** recursion/explicit stack; timestamps classify edges and support cycle detection, topological ordering, SCCs and articulation/bridge algorithms.
- For an undirected graph, DFS produces tree/back edges; a cross-edge classification from directed DFS should not be imported blindly.

METHOD

### Articulation point — Tarjan low-link test

During DFS, maintain `disc[u]` and `low[u]`. For a DFS tree edge u→v:

- If u is not the root: **u is an articulation point iff some child v has low[v] ≥ disc[u]**.
- If u is the DFS root: it is an articulation point iff it has at least **2 DFS children**.
- For an undirected graph, a bridge is detected by `low[v] > disc[u]`.

METHOD

### Greedy vs DP — selection test

Before choosing an algorithm, ask:

1. Does a locally optimal choice provably extend to a global optimum? → seek a **greedy-choice/exchange proof**.
2. Do many subproblems overlap and can an optimal solution be decomposed into optimal subsolutions? → use **DP**.
3. If neither property is apparent, do not force greedy or DP; consider graph search, divide-and-conquer, backtracking, or another formulation.

WORKED EXAMPLE

**Worked problem — double hashing**

**Table size m=11, $h_1(k)$=k mod 11, $h_2(k)$=7−(k mod 7). Insert 22, 33, 44 using open addressing.**

```
22: h₁=0, h₂=6 → slot 0.
33: h₁=0, h₂=2 → slot 0 occupied, then (0+2) mod 11 = 2.
44: h₁=0, h₂=5 → slot 0 occupied, then 5 is free.
Final occupied slots: 0→22, 2→33, 5→44.
```

WORKED EXAMPLE

**Worked problem — articulation point**

**DFS tree:** 1→2, 2→3, 3→4, with a back edge 4→2. Is 2 an articulation point?

```
disc[2] < disc[3].
The subtree rooted at 3 has a back edge to 2, so low[3] = disc[2].
Articulation condition for non-root 2 is low[3] ≥ disc[2].
Equality holds → vertex 2 is an articulation point (removing it separates 1 from the {3,4} part).
```

WORKED EXAMPLE

**Worked problem — heap sort**

**Build a max-heap from [4, 1, 3, 2].**

```
Start: [4,1,3,2].
Heapify index 1: 1 has child 2 → swap → [4,2,3,1].
Index 0 already has largest child 4.
Max-heap = [4,2,3,1].
Then repeatedly swap root with the last active element and heapify the reduced heap.
```

## 9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Master theorem:** $T(n)=aT(n/b)+f(n)$; compare $f(n)$ with $n^{\log_b a}$ before choosing the case.
- **Merge sort:** O(n log n) time and O(n) auxiliary space in the usual array implementation.
- Dijkstra is valid with non-negative edge weights; it is not a general shortest-path algorithm for negative edges.
- Union-Find with path compression + union by rank/size has near-constant amortized cost: $O(\alpha(n))$.

LAST-MINUTE · MUST REMEMBER

- For nested loops, derive the exact iteration count when possible; do not assume every nested loop is $O(n^2)$.
- Binary search requires a sorted search space; its standard comparison complexity is O(log n).
- Greedy algorithms require a proof of correctness, typically via greedy-choice and/or exchange arguments.
- DP is appropriate when subproblems overlap and the problem has optimal substructure.
- A DAG has a topological ordering iff it is acyclic; DFS finishing times can produce one.
- Hash-table performance depends on load factor and collision handling; worst-case search can still be O(n).
- **Heap sort:** build heap in O(n), then perform n−1 extract-max operations; O(n log n) time and O(1) auxiliary array space.
- BFS gives shortest paths in unweighted graphs; DFS supplies timestamps/low-link structure for cycle, SCC, bridge and articulation problems.

LAST-MINUTE · COMMON TRAPS

- Quicksort has $O(n^2)$ worst-case time; average/expected time is O(n log n) under standard randomisation assumptions.

LAST-MINUTE · GATE FAVOURITES

- Know the difference between MST algorithms, shortest-path algorithms, and traversal algorithms; they solve different problems.
- **Articulation:** non-root u is critical when some DFS child v has low[v] ≥ disc[u]; DFS root needs at least two children.